KV Cache and Memory Optimization

pig7selene

September 5, 2026

Abstract

The KV Cache makes incremental decoding possible by retaining attention keys and values for prior tokens, but it turns sequence length and concurrency into persistent memory demand. This chapter derives cache memory accounting from Transformer dimensions, distinguishes MHA, MQA, and GQA cache layouts, and explains allocation, fragmentation, paging, prefix sharing, and cache lifecycle. It then introduces cache-specific quantization as a memory-quality trade-off. The chapter develops the memory-management principles that later serving systems use without treating scheduling or model-weight quantization in depth.

1 Introduction

Chapter 19 established the two inference phases: Prefill processes a known prompt and constructs the KV Cache, while Decode reuses that state as it appends one token at a time. The cache eliminates a severe source of redundant computation, but it replaces that work with persistent request state. Unlike model weights, which are mostly fixed once a model is loaded, cache capacity depends on how many sequences are active and how long they become.

This distinction changes the resource problem of serving. A model that fits comfortably in accelerator memory at batch size one can admit far fewer concurrent long-context requests than its weight footprint alone suggests. The relevant question is not merely how many parameters the model has. It is how many layerwise key-value vectors must coexist, in which representation, for how long, and with what degree of sharing or waste.

The chapter uses the attention notation of Chapter 3. Let B be the number of active sequences, S their current cached length, L the number of Transformer blocks, g the number of key-value heads, and d_h the head dimension. Let b denote bytes per cached scalar. The discussion assumes a dense decoder-only Transformer and a cache that stores keys and values separately. Exact tensor strides, padding, metadata, and temporary workspaces vary by runtime, but the basic accounting does not.

2 KV Cache Structure and Memory Accounting

For one layer and one active sequence, the key cache and value cache each contain a vector for every cached position and every key-value head. Their shapes are

$$K_l, V_l \in \mathbb{R}^{S \times g \times d_h}, \quad l \in \{1, \ldots, L\}. \tag{1}$$

With a batch axis, each cache tensor has shape $B \times g \times S \times d_h$. One tensor contains $BSgd_h$ scalars. Since attention retains both keys and values, the persistent cache across all layers contains approximately

$$N_{\mathrm{KV}} = 2BSLgd_h \tag{2}$$

scalars and therefore occupies

$$M_{\mathrm{KV}} = 2BSLgd_h b \tag{3}$$

bytes. The factor two is not a constant to discard casually: the attention score needs historical keys, while the weighted attention output needs historical values. The remaining factors describe independent multiplicative pressure. Doubling active batch size, cached sequence length, layer count, key-value heads, head dimension, or byte width doubles the approximate persistent cache memory.

Equation 3 is an accounting model, not an allocator specification. A runtime may reserve tokens in blocks, pad a sequence, retain scaling metadata for quantized data, or keep temporary attention

buffers. The exact allocated memory can consequently exceed the ideal payload. Nevertheless, the equation identifies the dominant state whose linear growth makes long-context and high-concurrency serving difficult. Large-inference studies likewise separate fixed parameter memory from decoding-time KV state and its associated memory traffic [1].

Attention form	KV heads	Per-layer K or V shape	Both tensors, all layers
MHA	$g = h$	$B \times h \times S \times d_h$	$2BSLhd_hb$
GQA	$1 < g < h$	$B \times g \times S \times d_h$	$2BSLgd_hb$
MQA	$g = 1$	$B \times S \times d_h$	$2BSLd_hb$

Table 1: Ideal KV Cache payload for three attention organizations. Query-head count remains h; changing the number of key-value heads changes the persistent tensors that incremental attention reads.

2.1 MHA, MQA, and GQA

Multi-Head Attention (MHA) has one key and one value head for every query head, so $g = h$. Multi-Query Attention (MQA) keeps the h query heads but shares a single key head and a single value head across them, so $g = 1$. Grouped-Query Attention (GQA) lies between those endpoints: several query heads share each key-value head, with $1 < g < h$. Chapter 3 derives this head mapping and its attention computation.

The important memory consequence follows directly from Table 1. The number of query heads controls how many query rows form attention scores; it is not the same as the number of distinct historical key-value vectors. MQA and GQA reduce the latter without reducing the former in the same proportion. During Decode, this lowers cache capacity and the cache data read by attention. Shazeer introduced MQA as a decoding-oriented key-value sharing design [2], while Ainslie et al. frame GQA as an intermediate architecture that can retain much of MHA quality with much of MQA's inference benefit [3].

Head reduction is an architectural choice made when a model is trained or adapted. It is not a generic serving switch that can be applied to an arbitrary MHA checkpoint without changing its key and value projections. A serving runtime must discover the model's actual number of KV heads from the checkpoint configuration and use the corresponding cache shape. Treating query-head count as KV-head count is a common accounting error that can overestimate capacity by a large factor for GQA models.

3 Allocation, Fragmentation, and Paged KV Caches

The ideal payload in Equation 3 assumes that every allocated scalar represents a useful cached token. A simple static allocator violates that assumption when it reserves a contiguous region for each request's maximum permitted length. A short request then occupies capacity for tokens it will never generate; a request that terminates early leaves a partially unused reservation. Reserving for the maximum avoids reallocations, but it can reduce the number of requests admitted long before useful cache payload fills memory.

Dynamic contiguous allocation improves on static reservation by growing a request as it generates. It introduces a different problem. Many requests grow and terminate at different times, so free capacity becomes divided into non-adjacent holes. A later request may need a large contiguous region even when the total number of free bytes is sufficient. This **external fragmentation** is allocator-level waste: the memory exists, but its layout prevents an otherwise valid allocation.

Paged allocation avoids requiring one physical span per logical token sequence. Choose a fixed block size of P token positions. A logical cache sequence of length S then uses

$$n_{\text{pages}} = \text{ceil}\left(\frac{S}{P}\right) \tag{4}$$

logical pages per layer and per key-value tensor. A page table maps those logical pages to any available physical blocks. The final page may be partially occupied, but the unused capacity is bounded by fewer than P token positions per tensor rather than by the request's entire unused maximum length.

Figure 1: A paged KV Cache separates logical token order from physical block placement. Attention follows the logical page table; the runtime need not keep a request's cache in one contiguous physical allocation.

PagedAttention applies this virtual-memory-style organization to attention's key-value storage. A logical sequence is represented by fixed-size KV blocks that can occupy non-contiguous physical memory, while the attention computation consults the block mapping to read them in logical order. This organization limits reservation waste, accommodates variable generation lengths, and creates a natural unit for cache sharing. Kwon et al. introduced PagedAttention and the vLLM serving system to address fragmentation and redundant KV duplication under high-throughput serving [4].

Paging does not make cache memory free. Every live logical token still needs key and value storage, and page tables, block rounding, and allocator metadata have costs. The benefit is better utilization of the available capacity and a representation in which growing, terminating, and shared sequences need not be reshaped into a single contiguous buffer. Scheduling policy and continuous batching decide which requests advance; this chapter concerns the cache representation they manage.

4 Reuse, Prefix Sharing, and Cache Lifecycle

Three forms of reuse are often conflated. **KV reuse within one generation** is the ordinary Decode behavior from Chapter 19: a request reuses its own past keys and values after each new token. **Prefix caching** records the Prefill result for an already processed token prefix so that a later request with exactly that prefix can begin after it. **Prefix sharing** refers to a memory representation in which two or more live requests reference the same physical KV blocks for an identical prefix. Prefix caching can enable prefix sharing, but a cache lookup and a shared physical allocation are not identical mechanisms.

Suppose R requests share a tokenized prefix of length S_p and each has a distinct suffix of length S_u. Ignoring block rounding and metadata, storing every prefix separately costs a quantity proportional to

$$R(S_p + S_u), \tag{5}$$

whereas physically sharing the immutable prefix blocks costs a quantity proportional to

$$S_p + RS_u. \tag{6}$$

The difference is $(R-1)S_p$ token positions of cache payload per layer and per key-value tensor. Common system prompts, shared document prefixes, and repeated few-shot exemplars can therefore create meaningful reuse opportunities. The equality test must operate on the exact serialized token prefix and the same model revision, tokenizer, positional convention, cache dtype, and relevant attention configuration. Semantic similarity is insufficient: two similar prompts with different token IDs cannot share the same computed keys and values.

Cache lifetime has four operational phases. A request first allocates or acquires blocks during Prefill. It grows by appending blocks during Decode. Its blocks can become reusable after the request terminates, and a retained prefix may remain available for a future compatible request. Eviction removes retained state to recover capacity. Reference counting or an equivalent ownership rule is essential when blocks are shared: one request ending must not invalidate a prefix still referenced by another request.

4.1 Long Context and Eviction

For an unbounded generation, the cache grows with S in Equation 3 until it reaches the model's context limit or available memory. Long context makes the cache expensive in two ways. It consumes persistent capacity, and later Decode steps must read more historical keys and values. High concurrency compounds both effects because all active sequences contribute to the batch factor B.

An eviction policy bounds cache capacity by discarding some historical entries. This is not a transparent allocator optimization: discarded keys and values are no longer available to attention, so it changes the effective context seen by the model. A sliding window preserves recent tokens but may lose an early instruction or relevant document. More selective policies try to retain entries judged important as well as recent ones. H2O, for example, studies an eviction policy that balances recent tokens with attention heavy hitters [5]. Such policies are model- and task-dependent approximations, not a substitute for preserving the full declared context.

The cache owner should therefore state its eviction semantics explicitly: capacity bound, retained-window policy, selection criterion, whether eviction occurs per layer or globally, and how the resulting effective context is reported. An application that claims a certain context length while silently evicting most of it is describing a different capability from full-context attention.

5 KV Cache Quantization

KV Cache quantization reduces the byte width b in Equation 3 rather than changing model architecture. If a reference cache uses b_{ref} bytes per scalar and a quantized cache uses an effective b_{quant} bytes, the ideal payload ratio is

$$\frac{M_{\text{KV}}^{\text{quant}}}{M_{\text{KV}}^{\text{ref}}} \approx \frac{b_{\text{quant}}}{b_{\text{ref}}}. \tag{7}$$

The word **effective** matters. Low-bit blocks also require scales, zero-points, group metadata, alignment, and sometimes a recent full-precision region. Dequantization or mixed-precision attention kernels can add compute and memory traffic. The realized saving is consequently smaller or differently shaped than the scalar byte ratio alone.

Keys and values need not have identical quantization behavior. Keys affect attention scores before Softmax; values affect the weighted accumulation after attention probabilities have been formed. Quantization error can therefore alter both which positions receive attention and what information is aggregated from those positions. KIVI studies cache-specific asymmetric quantization and reports that its key and value cache distributions favor different grouping choices, illustrating why a single generic quantization rule need not be equally suitable for both tensors [6].

The practical trade-off is not simply memory against average perplexity. Cache quantization should be evaluated under the intended context lengths, output lengths, sampling policy, batch concurrency, and task distribution. A configuration that preserves short-answer quality may fail a retrieval-heavy long-context task, and a memory reduction that enables a larger batch may add latency if dequantization becomes a bottleneck. General model-weight quantization changes the storage and arithmetic of learned parameters; it is a related but distinct topic reserved for the next chapter.

6 Implementation Contracts

The cache contract begins with model identity. The runtime must know layer count, query-head count, KV-head count, head dimension, cache dtype, positional convention, and the exact tokenizer and model revision used to construct a prefix. Its allocated shapes should agree with Equation 1 and its accounting dashboard should distinguish ideal payload from reserved blocks, metadata, temporary workspaces, and model weights.

For paged storage, tests should verify that a logical page table produces the same attention output as a contiguous reference layout for the same prefix. They should exercise page growth at a block

boundary, requests with different termination lengths, block reuse after termination, and shared-prefix reference counts. A cancellation path must release only blocks whose last reader has departed. Metrics should report cache utilization and fragmentation separately from total allocated bytes; otherwise a full allocator can be mistaken for a full useful cache.

For prefix reuse, the lookup key must include the full token prefix and all state that affects cached keys and values. For cache quantization, the contract must define bit width, group shape, scale and zero-point representation, residual full-precision policy, dequantization kernel, and accepted numerical tolerance. Tests should compare quantized and reference cache logits or attention outputs on declared lengths and tasks, not merely verify that allocation succeeds. These conditions make a memory optimization auditable rather than an opaque reduction in reported bytes.

7 Summary

The KV Cache stores two persistent attention tensors for every cached position, layer, KV head, and active sequence. Its approximate footprint, $2BSLgd_hb$, explains why cache state becomes a first-order serving constraint under long contexts and high concurrency. MQA and GQA reduce the number of stored key-value heads while preserving the query-head organization that forms attention scores.

Memory optimization is therefore a representation and lifecycle problem as well as a byte-count problem. Paged KV Caches replace maximum-length contiguous reservations with fixed logical pages mapped to available physical blocks. Prefix sharing avoids duplicating immutable common prefixes, while explicit lifecycle and eviction rules determine when blocks may be retained or reclaimed. Cache quantization reduces bytes per element but can perturb attention scores and outputs, so its quality and performance must be tested in the actual serving regime. These principles prepare the later study of weight quantization, scheduling, and full serving systems.

References

[1] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," *arXiv preprint arXiv:2211.05102*, 2022, doi: 10.48550/arXiv.2211.05102.

[2] N. Shazeer, "Fast Transformer Decoding: One Write-Head Is All You Need," *arXiv preprint arXiv:1911.02150*, 2019, [Online]. Available: https://arxiv.org/abs/1911.02150

[3] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 4895–4901. doi: 10.18653/v1/2023.emnlp-main.298.

[4] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, Association for Computing Machinery, 2023, pp. 611–626. doi: 10.1145/3600006.3613165.

[5] Z. Zhang *et al.*, "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in *Advances in Neural Information Processing Systems 36*, Curran Associates, Inc., 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/6ceefa7b15572587b78ecfcebb2827f8-Abstract-Conference.html

[6] Z. Liu *et al.*, "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," *arXiv preprint arXiv:2402.02750*, 2024, doi: 10.48550/arXiv.2402.02750.